Weighted graphs and shortest paths

In a finite directed weighted graph, path cost sums edge weights. If a source-reachable negative cycle can also reach v, arbitrarily many repetitions make walk cost to v unbounded below. Vertices unreachable from that cycle are not affected. Dijkstra's general correctness premise is nonnegative weights on all source-reachable edges.

Tentative d[v] is an upper bound and π[v] a predecessor. Initialize d[s]=0,others infinity, and predecessors empty. DAGs allow topological relaxation; general negative weights without reachable negative cycles require Bellman–Ford.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra's invariant and proof

Extract the unfinalized minimum d[u]. Finalized distances are exact; finite tentative values have realizing source paths, and unreachable vertices retain infinity and no predecessor. Relax u→v by $d[v] = \min(d[v], d[u] + w(u, v))$.

Suppose dist(s, u)<d[u]. On a shortest path, take its first unfinalized x with finalized predecessor y. Earlier relaxation gave d[x]≤dist(s, y)+w(y, x)=dist(s, x). Nonnegativity implies dist(s, x)≤dist(s, u) <d[u],contradicting minimum extraction. If x=s, initial d[s]=0 already prevents the error.

Arrays cost $O(n^2+m)$. A binary heap costs O((n+m)log n). A Fibonacci heap has O(n) initialization/insertion, n amortized O(log n) extractions, m successful amortized O(1) decrease-keys, and O(m) relaxations, for O(m+n log n).

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Complete Dijkstra trace

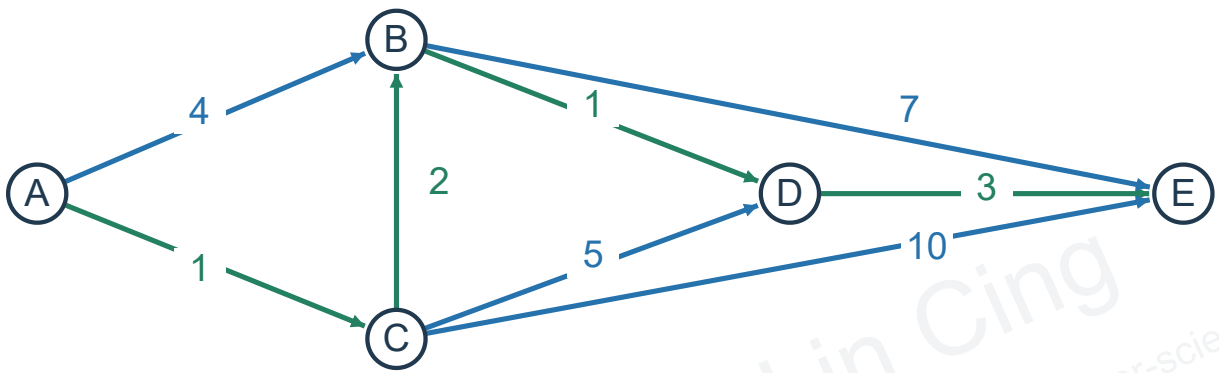
Use A→B:4,A→C:1,C→B:2,B→D:1,C→D:5,D→E:3,C→E:10,B→E:7,source A.

Finalized this round	A	B	C	D	E	Improved predecessors
Initial	0	∞	∞	∞	∞	None
A	0	4	1	∞	∞	B←A, C←A
C	0	3	1	6	11	B←C, D←C, E←C
B	0	3	1	4	10	D←B, E←B
D	0	3	1	4	7	E←D
E	0	3	1	4	7	None

Backtrack E←D←B←C←A to obtain A–C–B–D–E, cost 1+2+1+3=7. Update only on strict improvement. Lazy heaps with duplicate keys must skip stale entries rather than finalize them again.

Edge weights differ from total distances

Dijkstra: arc weights,final distances,and predecessor tree



Finalized: A,C,B,D,E

d(A,B,C,D,E) = (0,3,1,4,7)

Path to E:

A → C → B → D → E

Cost 1+2+1+3 = 7

Arc labels are weights; green arcs form the predecessor tree. The right table gives distances.

C→D has weight five, but d[D]=4 because C→B→D costs three plus A→C's one. An individual arc weight is not the entire distance to its destination.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Distance certificates and negative edges

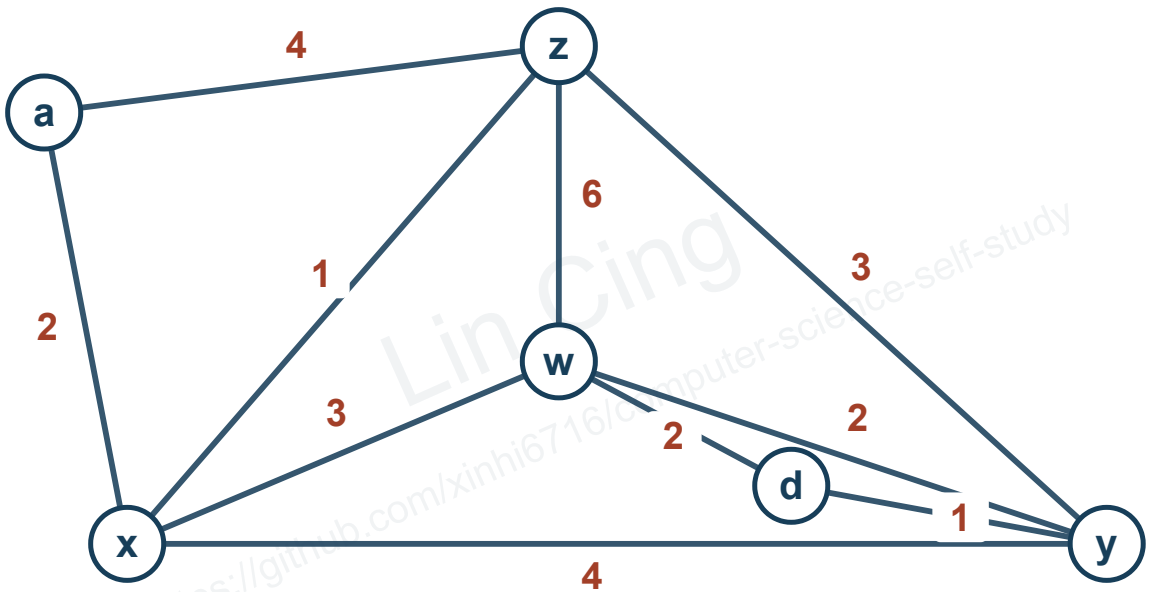
A certificate has $d[s]=0$,inequality $d[v] \leq d[u]+w(u, v)$ for every reachable arc, and a predecessor path realizing each finite $d[v]$. Applying inequalities along any source path makes $d[v]$ no larger than its cost; the realizing path attains that bound, proving exactness. Graph search separately certifies unreachable vertices. Extracting infinity permits stopping. Do not change finalized predecessors, avoiding unnecessary zero-weight parent cycles.

For $s \to a:2, s \to b:1, a \to b:-2$,there is no negative cycle, but finalizing $b=1$ misses true distance zero. Bellman–Ford's round k, reading previous-round distances, computes minimum walk cost with at most k edges. Without reachable negative cycles, a shortest walk removes nonnegative cycles to a path of at most $n-1$ edges. Thus $n-1$ rounds suffice; improvement in round n identifies a reachable negative cycle. Search its downstream vertices to identify affected distances, rather than declaring every vertex unbounded.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One graph: shortest paths and a minimum spanning tree

The graph is finite, undirected, and has the nonnegative edge weights shown. Run Dijkstra from a.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dijkstra: every round and relaxation

Settle equal-distance vertices in alphabetical order. Update a predecessor **only for a strict improvement**; retain it on equal distance. Kruskal processes equal-weight edges in alphabetical order of their endpoints.

Distances are listed in the fixed order a, x, z, w, y, d. An unsuccessful relaxation is still checked.

Settled	Attempts to unsettled vertices	Distances (a, x, z, w, y, d) afterward
a	x:2, z:4	(0,2,4,∞,∞,∞)
x	z:3<4, w:5, y:6	(0,2,3,5,6,∞)
z	w:9>5, y:6=6	(0,2,3,5,6,∞)
w	y:7>6, d:7	(0,2,3,5,6,7)
y	d:7=7	(0,2,3,5,6,7)
d	No unsettled neighbor	(0,2,3,5,6,7)

Kruskal: merge distinct components

For equal weights, order edges alphabetically by their sorted endpoints. Stop after selecting five edges.

Edge	Weight	Decision and reason
dy	1	Select; merge d and y
xz	1	Select; merge x and z
ax	2	Select; add a to the xz component
dw	2	Select; add w to the dy component
wy	2	Skip; w and y are already connected
wx	3	Select; merge the two components to finish

The selected set $\{dy, xz, ax, dw, wx\}$ has total weight $1 + 1 + 2 + 2 + 3 = 9$. Each selected edge crosses two current components and is minimum-weight across that component's cut in the sorted order, so the cut property certifies its safety.

Compare objectives and certificates

The Dijkstra predecessor tree is $\{ax, xz, xw, xy, wd\}$, with total edge weight $2 + 1 + 3 + 4 + 2 = 12$. Its distances from a, in the stated order, are $0, 2, 3, 5, 6, 7$. The Kruskal tree weighs 9, but its path $a \to x \to w \to d \to y$ has length $2 + 3 + 2 + 1 = 8$, greater than the shortest distance 6.

Strict improvements give predecessors $x \leftarrow a, z \leftarrow x, w \leftarrow x, y \leftarrow x, d \leftarrow w$; settlement order is a, x, z, w, y, d. For instance, $a \to x \to z \to y$ and $a \to x \to y$ both have length 6, so y retains predecessor x under the tie rule.

Both are spanning trees. A shortest-path tree minimizes each root-to-vertex distance; a minimum spanning tree minimizes the **sum of selected edge weights**. Ties can produce other valid trees, so this tie-breaking result is not claimed to be unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kruskal and the cut property

Sort undirected edges by weight and accept an edge exactly when it creates no cycle. For any nontrivial cut, each lightest crossing edge belongs to some MST. If an MST T omits such e, the unique T-path between e's endpoints includes a crossing f. Adding e makes a unique cycle; delete that f to preserve a tree. Since $w(f) \geq w(e)$,the replacement has no greater total weight and hence is also minimum.

For AB:2,BC:1,CA:4 plus CD:3,Kruskal takes BC, AB, CD, total six. Ties may permit multiple MSTs with the same minimum total.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prim's invariant and the cycle property

Prim grows one tree by choosing the lightest edge across its current vertex cut. Its accepted edges remain contained in some MST. Kruskal maintains the analogous extendable forest invariant.

A uniquely heaviest edge on a cycle belongs to no MST: delete it from a proposed MST, and replace it across that cut by a lighter cycle edge, strictly improving weight. If the maximum ties, some MST omits a specified heaviest edge, but not necessarily every MST. Disconnected graphs have no spanning tree; detect connectivity or output a minimum spanning forest.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Flow-network model

A finite directed network has distinct source s, sink t, and capacities $c(e) \geq 0$. Each original arc has nonnegative flow $0 \leq f(e) \leq c(e)$. Every internal vertex conserves inflow/outflow. Flow value is source outflow minus source inflow.

An undirected connection modeled as two directed arcs differs from a shared capacity; specify which. Antiparallel original arcs remain distinct records in residual construction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Residual records and augmentation

For each original ID $e=(u, v)$,create forward $e^{+}:u\to v$ of capacity $c(e)-f(e)$,and reverse $e^{-}:v\to u$ of capacity $f(e)$. The latter permits cancellation of existing flow.

An augmenting s-to-t residual path with positive bottleneck Δ increases original flow on forward records and decreases it on reverse records, preserving conservation and capacity limits. Same-direction records may have different origins and must not overwrite one another. Implementations normally store paired residual capacities and update both. A net-flow convention requires its own conversion. Residual paths represent possible adjustments, not necessarily original directed transport paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Max-flow min-cut

A cut partitions V into S, T with s∈S, t∈T, and counts only original S→T capacities. Summing conservation over S cancels internal arcs:

$$|f| = \sum_{S \to T} f(e) - \sum_{T \to S} f(e) \leq \sum_{S \to T} c(e) = c(S, T).$$

If no augmenting path remains, take S as residual-source-reachable vertices. Then t∉S. Every original S→T arc has zero forward residual and is saturated. Every original T→S arc has zero flow, because its reverse residual would otherwise cross from S to T. Hence |f|=c(S, T),certifying both optimums. The forward residual of an original T→S arc may remain positive; do not say all reverse-crossing records vanish.

Integer capacities and integer bottlenecks from zero flow raise value by at least one per augmentation, with a finite cut bound, so terminate at an integral optimum. Arbitrary real capacities with arbitrary path selection need not terminate in finitely many steps. Edmonds–Karp's shortest-edge-count paths give polynomially many augmentations. Minimum cuts may differ; maximum value is unique.

Bipartite matching and Hall's theorem

A matching in $G=(L \cup R, E)$ has no shared endpoints. Saturating L covers every left vertex; perfect means every vertex on both sides. If $|L|<|R|$,saturating L is not perfect. $N(S)$ is the full right neighborhood of $S \subseteq L$.

Hall's theorem gives a matching saturating L iff $|N(S)| \geq |S|$ for every $S \subseteq L$. Necessity follows because matched neighbors are distinct. For sufficiency, take maximum M and search from all unmatched left vertices along nonmatching $L \to R$ and matching $R \to L$ edges. Reaching an unmatched right vertex would augment, contradiction. Otherwise reachable sets S, T have $N(S)=T$, each T vertex matched into S, and S additionally contains an unmatched starting point. Thus $|N(S)|=|T|<|S|$,contradicting Hall. This is a global subset condition; algorithms search alternating paths instead of enumerating subsets.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Matching as unit-capacity flow

Connect s to every left vertex, orient original edges L→R, and connect each right vertex to t, all capacity one. An integral maximum flow selects a matching, with matching size equal to value. Kőnig equates maximum bipartite matching and minimum vertex-cover sizes. Weighted assignment additionally needs cost optimization, not merely cardinality. Integrality is guaranteed for integer-capacity optima; arbitrary real feasible flows need not be integral.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Common modeling errors

Negative edges invalidate Dijkstra's general theorem, as the s, a, b example shows. Omitting Kruskal's cycle test can produce a cycle and disconnection even after n−1 choices. Cut capacity includes forward capacities only. Singleton-neighborhood checks do not establish Hall. Maximum matching can leave unmatched vertices, and perfect requires both sides. Model assumptions determine which greedy argument applies.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Safe cuts must respect the selected forest

Assume F lies in some MST. A cut respects F when no F-edge crosses it. For its lightest crossing e, take an MST T containing F. If $e \notin T$, adding it creates a cycle with another crossing f. Respect implies $f \notin F$, so replacing f by e preserves all F edges and minimum cost. Thus $F \cup \{e\}$ remains extendable.

For Kruskal's next edge between different components, take one component as S. Any lighter previously scanned crossing edge could not have been rejected: rejection would have joined endpoints already in one component, which would still lie together now. Hence e is lightest for this cut. Prim chooses its current S's lightest crossing edge directly. Start at empty F and preserve the invariant until $n-1$ connected edges form an MST. Negative or zero MST weights are allowed; the exchange proof uses order comparisons, not Dijkstra's nonnegativity premise.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Kruskal and Prim on the same graph

Use AB:1,AC:4,BC:2,BD:5,CD:3.

Edge	Decision	Components afterward
AB:1	Accept	{A, B},{C},{D}
BC:2	Accept	{A, B, C},{D}
CD:3	Accept	{A, B, C, D}
AC:4	Reject cycle A–B–C–A	Unchanged
BD:5	Reject cycle B–C–D–B	Unchanged

https://github.com/xinhi6716/computer-science-self-study

Prim candidates and algorithm costs

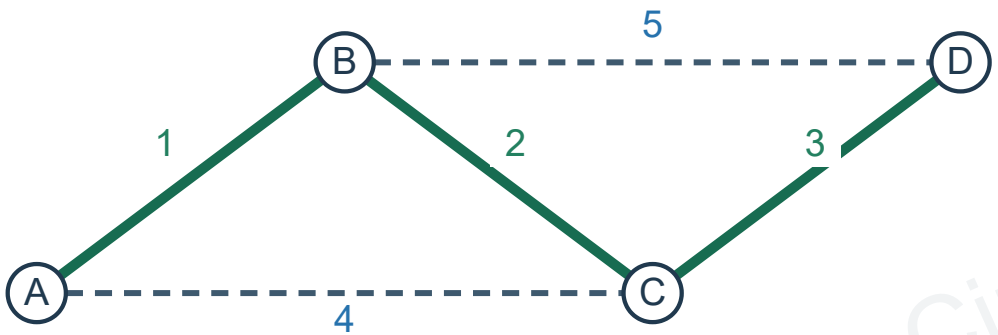
One can stop after three acceptances;remaining rows display cycle checks. Prim at A sees {AB:1,AC:4},then {BC:2,AC:4,BD:5},then {CD:3,BD:5}. It picks AB, BC, CD, total six. The final cut has both CD and BD.

Sorting plus DSU costs $O(n+m \log m+m\alpha(n))$,including initialization even with isolates/disconnection. Binary-heap Prim costs $O((n+m)\log n)$,array Prim $O(n^2+m)$. Disconnected inputs yield one minimum tree per component, not invented edges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Selected edges and every cut candidate

MST: choose safe cut edges, not source distances



Accept AB → BC → CD

Total weight 1+2+3 = 6

Final cut: {A,B,C} | {D}

Candidates CD:3, BD:5; choose CD.

Thick solid: selected edges. Dashed: other edges. BD crosses the final cut.

CD is uniquely lightest across the final cut and therefore lies in every MST. If omitted, the tree's cut-crossing path must use heavier BD; exchanging strictly improves it. This every-MST conclusion requires unique lightness, not a tied minimum.

Residual IDs and reverse updates

Each original e=(u, v, c) has its own paired forward residual c and reverse residual zero, with reciprocal reverse indices. Sending Δ subtracts it from the traversed record and adds it to its pair. Reverse use cancels old flow rather than making negative capacity.

Antiparallel original arcs require four records, two pairs. A reverse record of one is not the other's forward record. Aggregating same-direction residual capacity for search gives c(u, v)−f(u, v)+f(v, u),but reconstruction still needs origins. Nonnegative original-arc flow and skew-symmetric net flow have different capacity/value conventions and cannot be mixed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A complete cancellation augmentation

All original capacities are one on sa, sb, ab, at, bt. Sending one along s–a–b–t gives

$$(f_{sa}, f_{sb}, f_{ab}, f_{at}, f_{bt}) = (1, 0, 1, 0, 1).$$

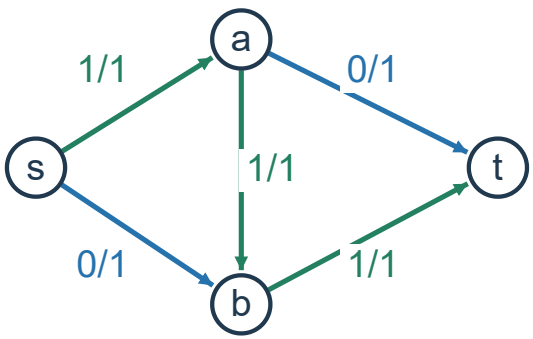
Though sa and bt are saturated, the residual path s–b–a–t exists, using the reverse of ab. Bottleneck one changes the vector to (1,1,0,1,1),raising value from one to two and canceling ab. It does not create original ba transport. Cut {s} has capacity two, certifying optimality. Searching only unsaturated original arcs would wrongly stop at one.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

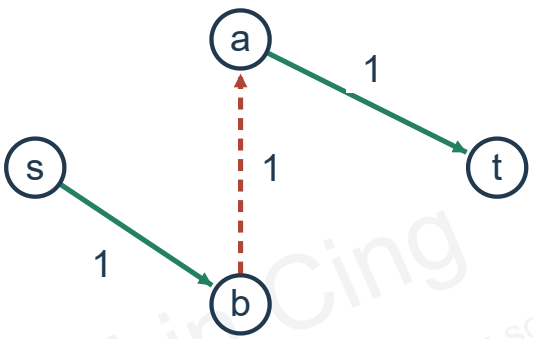
Reverse residual arcs change original flows

Residual augmentation: before,action,after; canceling old flow

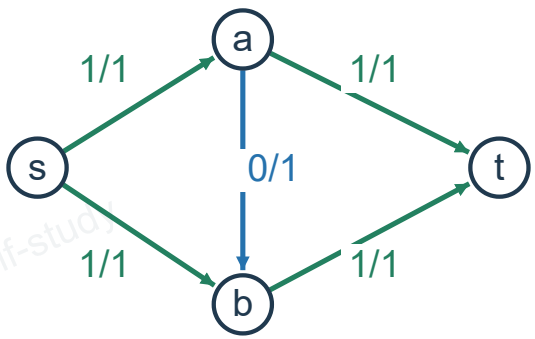
Before: original f/c,|f|=1



Action: augmenting path,Δ=1



After: original f/c,|f|=2



Middle labels are residual capacities. Red b→a cancels a→b; afterward a→b is 0/1.

Left and right diagrams show original f/c. The center shows only the three traversed residual records, labeled by residual capacities. Original ab:1/1 creates reverse ba residual one; afterward original ab is 0/1. Vertices and original arcs retain the same positions across states.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Edmonds–Karp's finite augmentation bound

BFS positive residual records to find a minimum-edge-count s-to-t path. If none exists, output flow and reachable cut. Otherwise backtrack predecessor records for the bottleneck, then update their pairs. BFS edge lengths are one, unrelated to capacity size.

Let $d(v)$ be old residual BFS distances. Newly positive records can only reverse path edges; a new $u \to v$ has old $d(v) = d(u) - 1$. Existing records satisfy $d(v) \leq d(u) + 1$. Summing along any new residual path shows old distance never exceeds its new length, so BFS distances cannot decrease. A newly reachable formerly unreachable vertex would have a first such entry on its path, contradicting the same inequalities.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Edmonds–Karp: bounding saturations

When u→v becomes saturated as a bottleneck,d(v)=d(u)+1. Before it can become a bottleneck again, reverse v→u flow must occur. At that later time,

$$d_{\text{later}}(u) = d_{\text{later}}(v) + 1 \geq d_{\text{old}}(v) + 1 = d_{\text{old}}(u) + 2.$$

Finite reachable distances are at most n−1,so each record is a bottleneck O(n) times. O(m) records and at least one saturated per augmentation give O(nm) augmentations. Excluding irrelevant isolates, BFS costs O(m),giving $O(nm^2)$ core time plus O(n+m) input setup. Exact arithmetic is assumed; integrality is not. Floating tolerances need a separate specification.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maximum, maximal, and augmenting paths

Maximum means globally most edges; maximal means no edge can simply be added. With a1,a2,b1,M= {a1} is maximal but alternating path b–1–a–2 flips to {b1,a2},increasing size.

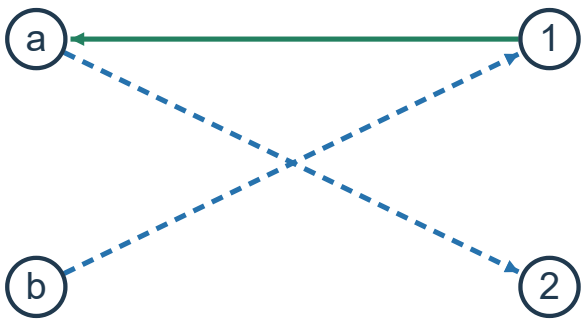
Berge's criterion says M is maximum iff no augmenting path exists. A path gives improvement by symmetric difference. Conversely, for larger M',$M \triangle M'$ has degree at most two and splits into alternating paths and even cycles. Cycles and even paths have equal counts from each matching. Since M' is larger, some path has one extra M' edge and both endpoints unmatched in M:an augmenting path. Direct additions alone do not suffice.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

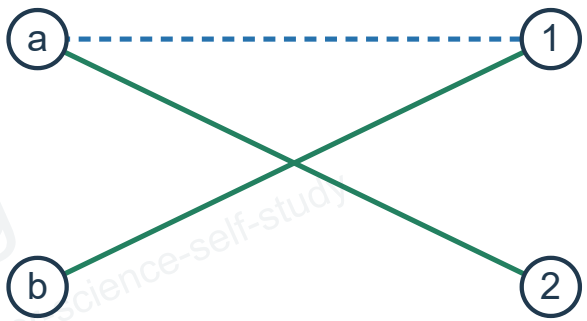
Flipping one alternating path

Alternating path b–1–a–2: flip edges to raise matching size from 1 to 2

Before M={a1}: search alternating edges



After M′={b1,a2}



Green solid edges are matched; blue dashed edges unmatched. Crossings are not vertices.

Traverse matched a1 backward from 1 to a to cancel it; the other edges go left to right. Each internal vertex loses one matched edge and gains one; both unmatched endpoints gain one. The result is a matching with exactly one additional edge.

Lin Cing

Constructive Kőnig theorem

For maximum M, search from unmatched left vertices along unmatched L→R and matched R→L edges. Let reachable sets be Z_L, Z_R. Every reachable right vertex is matched, or an augmenting path would exist. Set

$$C = (L \setminus Z_L) \cup Z_R.$$

It covers all edges. A left endpoint outside Z_L is covered directly. If inside, an unmatched edge reaches its right endpoint; for a matched edge, the right endpoint was the step reaching that matched left vertex. Unmatched starting vertices have no matched edge. Thus that right endpoint is in Z_R.

Every Z_R vertex is matched into Z_L; every left vertex outside Z_L is matched, otherwise it would start the search. These two cover parts count each M-edge exactly once, so |C|=|M|. Every cover must choose an endpoint for each disjoint matching edge, so no cover is smaller. Hence minimum cover equals maximum matching.

For {b1,a2},all L is saturated and reachability sets are empty, so C=L of size two. With only a1,b1,start at b and reach 1,a:Z_L={a, b},Z_R={1},giving minimum cover {1}.

Hall deficiency as a failure certificate

At most |N(S)| vertices in S can match, so any matching leaves at least |S|−|N(S)| unmatched left vertices. For maximum size ν,

$$|L| - \nu \geq \max_{S \subseteq L}(|S| - |N(S)|).$$

For the reverse inequality, use alternating reachable S=Z_L, T=Z_R of a maximum matching. T is matched into S, and S additionally contains exactly the k=|L|−ν unmatched left starts. Thus |S|−|T|=k, and N(S)=T, attaining equality. If k=0,the empty set attains zero.

For a, b, c adjacent only to 1,2,and d only to 3,subset {a, b, c} has deficiency one. Matching {a1,b2,d3} has size three and leaves exactly one left vertex, providing both feasibility and impossibility of improvement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Regular bipartite graphs

In a finite nonempty r-regular bipartite graph, $r \geq 1$,all $r|S|$ edges from S end in N(S),whose vertices each receive at most r edges. Thus $r|S| \leq r|N(S)|$ and Hall holds. Edge counting $r|L|=r|R|$ gives equal sides, so an L-saturating matching is perfect. At r=0 cancellation is invalid and isolated vertices need not have a perfect matching.

Delete a perfect matching; every degree drops by one. Repeating decomposes all edges into r disjoint perfect matchings. In $K_{2,2}$ use {a1,b2} and {a2,b1}. Coloring by matching gives r edge colors, with all incident colors distinct, and degree r requires at least r.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Certificates and complexity checks

Shortest paths use realizing paths plus arc inequalities; MSTs use safe cuts respecting the forest; maximum flows use equal-value cuts; maximum bipartite matchings use equal-size covers or no augmenting path. State representation, priority queue, input distribution, and worst/average cost before comparing speed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Optimization objectives are not interchangeable

MST minimizes total selected-tree weight, not all source-to-vertex tree distances. In triangle AB:2,BC:2,AC:3,the MST uses AB, BC, total four, but its A-to-C route costs four versus shortest distance three.

Maximum cardinality ignores assignment costs. With a1,b2 costing 100 each and a2,b1 costing one each, both perfect matchings have size two but costs 200 and two. Flow value alone cannot choose the cheaper; add cost minimization. Separate exercise and solution PDFs provide further examples.

References and attribution

Lin Cing, handwritten discrete mathematics notes. Ralph P. Grimaldi, Discrete and Combinatorial Mathematics, 5th ed., Ch. 13.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study